Contents

NumPy + Pandas — The ML-Ready Reference Book **1**
How this book is organized . . . 1
1. Mindmap Overview . . . 2
Quick Index . . . 3

PART 1 — NumPy **3**
1. Array Creation . . . 3
2. Attributes & Dtypes . . . 5
3. Indexing & Slicing . . . 6
4. Math & Broadcasting . . . 7
5. Aggregation & Stats . . . 9
6. Linear Algebra . . . 10
7. Reshape & Combine . . . 11
8. Random . . . 12
9. Sorting & Searching . . . 13
10. ML Utilities . . . 13

PART 2 — Pandas **15**
1. Creating Data . . . 15
2. Reading & Inspecting . . . 16
3. Selection & Indexing . . . 17
4. Data Cleaning . . . 19
5. Column Operations . . . 20
6. GroupBy & Aggregation . . . 22
7. Merging & Joining . . . 23
8. Sorting & Ranking . . . 24
9. DateTime . . . 25
10. Stats & Correlation . . . 25
11. ML Data Prep . . . 26

PART 3 — End-to-End Mini Pipeline **28**
Cheat Sheet — "I need to..." → method . . . 29
Where to go next . . . 30

NumPy + Pandas — The ML-Ready Reference Book

Intermediate → Advanced, code-verified, built for fast lookup.

Every snippet in this book was **actually executed** — the output blocks you see are real, not typed by hand. Copy-paste any block and it will run.

How this book is organized

1. **Mindmap Overview** — the whole mental model in one glance
2. **NumPy** — 10 sections, numerical computing foundation
3. **Pandas** — 11 sections, tabular data + ML preprocessing
4. **End-to-end cheat sheet** — a real mini ML pipeline using everything together

Use `Ctrl+F` / `Cmd+F` on any method name (e.g. `groupby`, `reshape`, `fillna`) to jump straight to it.

1. Mindmap Overview

Figure 1: NumPy and Pandas mindmap

One-line mental model: - **NumPy** = fast math on grids of numbers (no labels). Powers everything underneath. - **Pandas** = NumPy + row/column labels + missing-data handling + SQL-like operations. Built on top of NumPy.

Typical ML data flow: `pd.read_csv()` → clean (`dropna`/`fillna`) → explore (`describe`/`corr`/`groupby`) → encode (`get_dummies`) → convert to arrays (`.values` / `.to_numpy()`) → feed into NumPy-based math or a model (scikit-learn, etc).

Quick Index

NumPy sections

- 1. Array Creation
- 2. Attributes & Dtypes
- 3. Indexing & Slicing
- 4. Math & Broadcasting
- 5. Aggregation & Stats
- 6. Linear Algebra
- 7. Reshape & Combine
- 8. Random
- 9. Sorting & Searching
- 10. ML Utilities

Pandas sections

- 1. Creating Data
- 2. Reading & Inspecting
- 3. Selection & Indexing
- 4. Data Cleaning
- 5. Column Operations
- 6. GroupBy & Aggregation
- 7. Merging & Joining
- 8. Sorting & Ranking
- 9. DateTime
- 10. Stats & Correlation
- 11. ML Data Prep

PART 1 — NumPy

1. Array Creation

np.array() — from Python list

The entry point into NumPy. Converts a Python list (or list of lists) into an ndarray so it gets fast vectorized math instead of slow Python loops. Almost every array you touch starts here or is derived from one that did.

```
import numpy as np

a = np.array([1, 2, 3, 4])
b = np.array([[1, 2, 3], [4, 5, 6]])
```

```
print(a)
print(b)
print(type(a))
```

Output:

```
[1 2 3 4]
[[1 2 3]
 [4 5 6]]
<class 'numpy.ndarray'>
```

np.zeros() / np.ones() / np.full()

Pre-allocate an array of a given shape filled with 0, 1, or a custom value. Used to initialize weight matrices, accumulators, or placeholder arrays before filling them in a loop.

```
import numpy as np

print(np.zeros((2, 3)))
print(np.ones((2, 3)))
print(np.full((2, 2), 7))
```

Output:

```
[[0. 0. 0.]
 [0. 0. 0.]]
[[1. 1. 1.]
 [1. 1. 1.]]
[[7 7]
 [7 7]]
```

np.eye() — identity matrix

Creates an identity matrix (1s on the diagonal, 0s elsewhere). Shows up in linear algebra — e.g. as the starting point for regularization terms (ridge regression uses `lambda * np.eye(n)`).

```
import numpy as np

print(np.eye(3))
```

Output:

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

np.arange() / np.linspace()

arange generates values by step size (like Python's range but returns an array). `linspace` generates a fixed number of evenly spaced values between two endpoints — very common for plotting x-axes or sweeping a hyperparameter.

```
import numpy as np

print(np.arange(0, 10, 2))       # start, stop, step
print(np.linspace(0, 1, 5))      # start, stop, num points
```

Output:

```
[0 2 4 6 8]
[0.   0.25 0.5  0.75 1.  ]
```

np.zeros_like() / np.ones_like()

Creates a new array of zeros/ones with the same shape and dtype as an existing array. Handy when you need a same-shaped buffer without hardcoding dimensions.

```
import numpy as np

a = np.array([[1, 2], [3, 4]])
print(np.zeros_like(a))
print(np.ones_like(a))
```

Output:

```
[[0 0]
 [0 0]]
[[1 1]
 [1 1]]
```

2. Attributes & Dtypes

shape, ndim, size, dtype

The four properties you check first when debugging shape errors in ML code. shape tells you rows/cols, `ndim` the number of dimensions, `size` the total element count, `dtype` the data type (affects memory and precision).

```
import numpy as np

a = np.array([[1, 2, 3], [4, 5, 6]])
print("shape:", a.shape)
print("ndim :", a.ndim)
print("size :", a.size)
print("dtype:", a.dtype)
```

Output:

```
shape: (2, 3)
ndim : 2
size : 6
dtype: int64
```

astype() — change dtype

Converts an array's data type (e.g. float to int, or float64 to float32 to save memory). Common when a model expects a specific dtype, or when you want to shrink memory footprint for large datasets.

```
import numpy as np

a = np.array([1.7, 2.3, 3.9])
b = a.astype(np.int32)
```

```
print(a, a.dtype)
print(b, b.dtype)
```

Output:

```
[1.7 2.3 3.9] float64
[1 2 3] int32
```

reshape() / flatten() / ravel()

reshape changes an array's dimensions without changing its data — essential when feeding data into models that expect a specific shape (e.g. images into `(batch, height, width, channels)`). `flatten`/`ravel` collapse any shape back to 1D.

```
import numpy as np

a = np.arange(12)
b = a.reshape(3, 4)
print(b)
print(b.flatten())   # returns a copy
print(b.ravel())     # returns a view (faster, no copy if possible)
```

Output:

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[ 0  1  2  3  4  5  6  7  8  9 10 11]
[ 0  1  2  3  4  5  6  7  8  9 10 11]
```

3. Indexing & Slicing

Basic slicing

Python-list-style slicing `[start:stop:step]` works on NumPy arrays too, and is used constantly to grab subsets of data — e.g. the last N rows, or every other sample.

```
import numpy as np

a = np.array([10, 20, 30, 40, 50])
print(a[1:4])       # elements 1,2,3
print(a[::-1])      # reversed
print(a[-2:])       # last two
```

Output:

```
[20 30 40]
[50 40 30 20 10]
[40 50]
```

2D indexing (rows, cols)

For 2D arrays (matrices), index as `array[row, col]`. `:` means "all" along that axis. This is how you pull out a single feature column, a single sample row, or a sub-block of a dataset matrix.

```
import numpy as np

a = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(a[1, 2])       # row 1, col 2
print(a[:, 1])        # entire column 1
print(a[0:2, 1:3])    # sub-matrix
```

Output:

```
6
[2 5 8]
[[2 3]
 [5 6]]
```

Boolean mask indexing (very common in ML)

Creates a True/False array from a condition, then uses it to filter or edit values in place. This is the core trick behind outlier removal, thresholding, and conditional cleaning of numeric data.

```
import numpy as np

a = np.array([1, -2, 3, -4, 5])
mask = a > 0
print(mask)
print(a[mask])            # keep only positive values
a[a < 0] = 0              # replace negatives with 0
print(a)
```

Output:

```
[ True False  True False  True]
[1 3 5]
[1 0 3 0 5]
```

Fancy indexing

Indexing with a list/array of positions instead of a slice — lets you pull an arbitrary, non-contiguous set of elements. Used e.g. to select specific rows chosen by a shuffled index array (train/test splitting).

```
import numpy as np

a = np.array([10, 20, 30, 40, 50])
idx = [0, 2, 4]
print(a[idx])
```

Output:

```
[10 30 50]
```

4. Math & Broadcasting

Elementwise arithmetic

Arithmetic operators on arrays act element-by-element automatically (no loop needed). This vectorization is why NumPy is fast and is the foundation of nearly every ML computation.

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([10, 20, 30])
print(a + b)
print(a * b)
print(b / a)
print(a ** 2)
```

Output:

```
[11 22 33]
[10 40 90]
[10. 10. 10.]
[1 4 9]
```

Broadcasting (scalar & shape mismatch)

NumPy's rule for combining arrays of different (but compatible) shapes without manually copying data — e.g. adding a single row to every row of a matrix. Understanding broadcasting is essential for writing correct, fast ML code without explicit loops.

```
import numpy as np

a = np.array([[1, 2, 3], [4, 5, 6]])
print(a + 10)                       # scalar broadcast

row = np.array([100, 200, 300])
print(a + row)                       # (2,3) + (3,) -> broadcasts across rows
```

Output:

```
[[11 12 13]
 [14 15 16]]
[[101 202 303]
 [104 205 306]]
```

Universal functions (ufuncs)

Fast, elementwise math functions (sqrt, exp, log, abs, sin, etc). These are the building blocks of loss functions, activation functions (e.g. np.exp for sigmoid/softmax), and feature transformations.

```
import numpy as np

a = np.array([1, 4, 9, 16])
print(np.sqrt(a))
print(np.exp(np.array([0, 1, 2])))
print(np.log(np.array([1, np.e, np.e**2])))
print(np.abs(np.array([-3, -1, 2])))
```

Output:

```
[1. 2. 3. 4.]
[1.         2.71828183 7.3890561 ]
[0. 1. 2.]
```

```
[3 1 2]
```

5. Aggregation & Stats

sum, mean, std, var, min, max

The basic descriptive statistics used to summarize a dataset or a feature column — mean/std in particular are needed for standardization, a near-universal ML preprocessing step.

```
import numpy as np

a = np.array([[1, 2, 3], [4, 5, 6]])
print("sum :", a.sum())
print("mean:", a.mean())
print("std :", a.std())
print("var :", a.var())
print("min :", a.min(), " max:", a.max())
```

Output:

```
sum : 21
mean: 3.5
std : 1.707825127659933
var : 2.9166666666666665
min : 1  max: 6
```

Axis-wise aggregation (row vs column)

`axis=0` aggregates down columns (per-feature), `axis=1` aggregates across rows (per-sample). Getting this right matters a lot in ML — you usually want per-feature mean/std, not a single global number.

```
import numpy as np

a = np.array([[1, 2, 3], [4, 5, 6]])
print("column sums (axis=0):", a.sum(axis=0))
print("row sums    (axis=1):", a.sum(axis=1))
```

Output:

```
column sums (axis=0): [5 7 9]
row sums    (axis=1): [ 6 15]
```

argmax / argmin / median

`argmax/argmin` return the index* of the largest/smallest value rather than the value itself — this is exactly how you turn a model's output probabilities into a predicted class label. `median` is a robust-to-outliers alternative to mean.*

```
import numpy as np

a = np.array([3, 7, 1, 9, 4])
print("argmax:", np.argmax(a))
print("argmin:", np.argmin(a))
print("median:", np.median(a))
```

Output:

```
argmax: 3
argmin: 2
median: 4.0
```

6. Linear Algebra

dot product / matrix multiplication

Matrix multiplication — the single most-used operation in ML math, underlying linear regression, neural network layers (`X @ W + b`), and similarity computations.

```
import numpy as np

a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])
print(np.dot(a, b))
print(a @ b)          # same as np.dot for 2D arrays
```

Output:

```
[[19 22]
 [43 50]]
[[19 22]
 [43 50]]
```

transpose, inverse, determinant

`.T` flips rows/columns (needed constantly to make matrix shapes compatible for multiplication). `inv`/`det` solve systems of linear equations — the closed-form solution to linear regression uses the matrix inverse directly.

```
import numpy as np

a = np.array([[4, 7], [2, 6]])
print("Transpose:\n", a.T)
print("Inverse:\n", np.linalg.inv(a))
print("Determinant:", np.linalg.det(a))
```

Output:

```
Transpose:
 [[4 2]
 [7 6]]
Inverse:
 [[ 0.6 -0.7]
 [-0.2  0.4]]
Determinant: 10.000000000000002
```

eigenvalues / eigenvectors (used in PCA)

Decomposes a matrix into its principal directions and magnitudes. This is the math engine behind PCA (dimensionality reduction) — eigenvectors of the covariance matrix become the new axes you project data onto.

```
import numpy as np
```

```
a = np.array([[2, 0], [0, 3]])
vals, vecs = np.linalg.eig(a)
print("Eigenvalues :", vals)
print("Eigenvectors:\n", vecs)
```

Output:

```
Eigenvalues : [2. 3.]
Eigenvectors:
 [[1. 0.]
 [0. 1.]]
```

norm (used in regularization, distances)

Measures the "length" of a vector. L2 norm = Euclidean distance (used in KNN, clustering, and L2/Ridge regularization). L1 norm sums absolute values (used in Lasso regularization and outlier-robust losses).

```
import numpy as np

v = np.array([3, 4])
print("L2 norm:", np.linalg.norm(v))          # euclidean distance
print("L1 norm:", np.linalg.norm(v, ord=1))
```

Output:

```
L2 norm: 5.0
L1 norm: 7.0
```

7. Reshape & Combine

concatenate / stack

Combine multiple arrays into one. `concatenate` joins along an existing axis, `stack` creates a brand-new axis. Used to merge batches of data, or to assemble a feature matrix from separate feature arrays.

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(np.concatenate([a, b]))
print(np.stack([a, b]))          # new axis -> shape (2,3)
print(np.vstack([a, b]))
print(np.hstack([a, b]))
```

Output:

```
[1 2 3 4 5 6]
[[1 2 3]
 [4 5 6]]
[[1 2 3]
 [4 5 6]]
[1 2 3 4 5 6]
```

split arrays

The inverse of concatenation — breaks one array into several equal (or custom-sized) pieces. Useful for chunking a dataset for batch processing or cross-validation folds.

```
import numpy as np

a = np.arange(9)
parts = np.split(a, 3)
print(parts)
```

Output:

```
[array([0, 1, 2]), array([3, 4, 5]), array([6, 7, 8])]
```

8. Random

seed, rand, randn, randint

seed makes randomness reproducible — critical so your ML experiments give the same result every run. rand/randn/randint generate random numbers from uniform, normal, and integer distributions respectively — used for weight initialization and synthetic data.

```
import numpy as np

np.random.seed(42)          # reproducibility
print(np.random.rand(3))          # uniform [0,1)
print(np.random.randn(3))         # standard normal
print(np.random.randint(0, 10, size=5))
```

Output:

```
[0.37454012 0.95071431 0.73199394]
[-1.11188012  0.31890218  0.27904129]
[7 2 5 4 1]
```

choice, shuffle, permutation (train/test split building blocks)

choice samples random elements (with or without replacement — the basis of bootstrap sampling). permutation/shuffle randomly reorder data, which is exactly what happens under the hood in train_test_split.

```
import numpy as np

np.random.seed(0)
data = np.arange(10)
print(np.random.choice(data, size=3, replace=False))
print(np.random.permutation(data))     # shuffled copy

np.random.shuffle(data)                 # shuffles in-place
print(data)
```

Output:

```
[2 8 4]
[3 5 1 2 9 8 0 6 7 4]
[2 3 8 4 5 1 0 6 9 7]
```

9. Sorting & Searching

sort / argsort

sort returns values in order; argsort returns the indices* that would sort the array — more useful in ML since you often need to sort one array (e.g. predictions) while keeping another array (e.g. labels) aligned to it.*

```
import numpy as np

a = np.array([3, 1, 4, 1, 5, 9])
print(np.sort(a))
print(np.argsort(a))          # indices that would sort the array
```

Output:

```
[1 1 3 4 5 9]
[1 3 0 2 4 5]
```

where / unique / nonzero

where is a vectorized if/else, great for conditional feature engineering. unique finds distinct values (e.g. distinct class labels). nonzero returns indices of non-zero elements, often used with masks.

```
import numpy as np

a = np.array([1, -2, 3, -4, 5])
print(np.where(a > 0, a, 0))         # conditional replace (like a mini if-else)
print(np.unique(np.array([1, 2, 2, 3, 3, 3])))
print(np.nonzero(a))
```

Output:

```
[1 0 3 0 5]
[1 2 3]
(array([0, 1, 2, 3, 4]),)
```

10. ML Utilities

clip (bound values — e.g. gradient clipping)

Forces values into a min/max range. Used for gradient clipping (preventing exploding gradients in training) and for keeping predicted probabilities away from 0/1 to avoid log(0) errors in loss functions.

```
import numpy as np

a = np.array([-5, 0, 5, 10, 15])
print(np.clip(a, 0, 10))
```

Output:

```
[ 0  0  5 10 10]
```

isnan / nan_to_num (handling missing values)

Detects and replaces missing (NaN) values at the array level — the NumPy-side counterpart to Pandas' `isnull/fillna`, useful once your data has already been converted to arrays.

```
import numpy as np

a = np.array([1.0, np.nan, 3.0, np.nan])
print(np.isnan(a))
print(np.nan_to_num(a, nan=0.0))
```

Output:

```
[False  True False  True]
[1. 0. 3. 0.]
```

normalization example (min-max scaling from scratch)

Rescales values into a fixed [0, 1] range. Many algorithms (especially distance-based ones like KNN, or neural nets) train better/faster when features are on comparable scales.

```
import numpy as np

x = np.array([10, 20, 30, 40, 50], dtype=float)
x_norm = (x - x.min()) / (x.max() - x.min())
print(x_norm)
```

Output:

```
[0.   0.25 0.5  0.75 1.  ]
```

standardization example (z-score, used before most ML models)

Rescales a feature to have mean 0 and standard deviation 1 (a "z-score"). This is the single most common preprocessing step in ML — it's what `sklearn.preprocessing.StandardScaler` does internally.

```
import numpy as np

x = np.array([10, 20, 30, 40, 50], dtype=float)
x_std = (x - x.mean()) / x.std()
print(x_std)
print("new mean:", round(x_std.mean(), 5), " new std:", round(x_std.std(), 5))
```

Output:

```
[-1.41421356 -0.70710678  0.          0.70710678  1.41421356]
new mean: 0.0  new std: 1.0
```

PART 2 — Pandas

1. Creating Data

pd.Series() — 1D labeled array

A Series is a single labeled column of data (values + an index). It's the building block a DataFrame is made of — think of it as one column pulled out of a spreadsheet.

```
import pandas as pd

s = pd.Series([10, 20, 30], index=["a", "b", "c"])
print(s)
print(s["b"])
```

Output:

```
a    10
b    20
c    30
dtype: int64
20
```

pd.DataFrame() — from dict

The core Pandas object: a labeled 2D table (rows + named columns), like a spreadsheet or SQL table. Building one from a dict of column-name → values is the most common way to construct data manually or after loading it from another source.

```
import pandas as pd

df = pd.DataFrame({
    "name": ["Alice", "Bob", "Charlie"],
    "age": [25, 30, 35],
    "salary": [50000, 60000, 75000]
})
print(df)
```

Output:

```
      name  age  salary
0    Alice   25   50000
1      Bob   30   60000
2  Charlie   35   75000
```

pd.DataFrame() — from list of lists

Alternative construction when your data comes as rows (e.g. from a database cursor or CSV parsed manually) rather than columns — pass row data plus a `columns` list.

```
import pandas as pd

data = [[1, "A", 90], [2, "B", 85], [3, "C", 70]]
df = pd.DataFrame(data, columns=["id", "grade", "score"])
print(df)
```

Output:

```
   id grade  score
0   1     A     90
1   2     B     85
2   3     C     70
```

2. Reading & Inspecting

head(), tail(), shape, info(), describe()

The first things you run on any new dataset. head/tail peek at rows, shape gives (rows, cols), describe gives count/mean/std/min/max per numeric column — a fast sanity check before you do anything else.

```
import pandas as pd

df = pd.DataFrame({
    "name": ["Alice", "Bob", "Charlie", "David", "Eva"],
    "age": [25, 30, 35, 28, 40],
    "salary": [50000, 60000, 75000, 52000, 90000]
})

print(df.head(2))
print(df.shape)
print(df.describe())
```

Output:

```
    name  age  salary
0  Alice   25   50000
1    Bob   30   60000
(5, 3)
            age        salary
count   5.00000      5.000000
mean   31.60000  65400.000000
std     5.94138  16905.620367
min    25.00000  50000.000000
25%    28.00000  52000.000000
50%    30.00000  60000.000000
75%    35.00000  75000.000000
max    40.00000  90000.000000
```

to_csv() / read_csv()

read_csv is how almost every real ML project starts — loading a dataset from disk into a DataFrame. to_csv writes one back out, e.g. to save cleaned data or predictions.

```
import pandas as pd

df = pd.DataFrame({"a": [1, 2, 3], "b": [4, 5, 6]})
df.to_csv("sample.csv", index=False)

loaded = pd.read_csv("sample.csv")
print(loaded)
```

Output:

```
   a  b
0  1  4
1  2  5
2  3  6
```

columns, dtypes, index

Quick attribute checks: `columns` lists column names, `dtypes` shows each column's data type (important for spotting numbers accidentally stored as text), `index` shows the row labels.

```
import pandas as pd

df = pd.DataFrame({"a": [1, 2], "b": ["x", "y"]})
print("columns:", list(df.columns))
print("dtypes :\n", df.dtypes)
print("index  :", df.index)
```

Output:

```
columns: ['a', 'b']
dtypes :
 a    int64
b      str
dtype: object
index  : RangeIndex(start=0, stop=2, step=1)
```

3. Selection & Indexing

Column selection

The most basic operation — pick out one column (`df["col"]`, returns a Series) or several (`df[[...]]`, returns a DataFrame). Used constantly to build feature subsets.

```
import pandas as pd

df = pd.DataFrame({"name": ["Alice", "Bob"], "age": [25, 30], "salary": [50000, 60000]})
print(df["age"])                 # single column -> Series
print(df[["name", "salary"]])    # multiple columns -> DataFrame
```

Output:

```
0    25
1    30
Name: age, dtype: int64
    name  salary
0  Alice   50000
1    Bob   60000
```

loc[] — label based

Selects rows/columns by their actual label (index name or column name), not position. Use `loc` when you know the row name you want.

```
import pandas as pd

df = pd.DataFrame({"age": [25, 30, 35]}, index=["Alice", "Bob", "Charlie"])
```

```
print(df.loc["Bob"])
print(df.loc["Alice":"Bob"])       # inclusive of end label
```

Output:

```
age    30
Name: Bob, dtype: int64
       age
Alice   25
Bob     30
```

iloc[] — position based

Selects rows/columns by integer position (like NumPy/list indexing), regardless of labels. Use `iloc` when you want "the first N rows" or "the 3rd column" without caring about their names.

```
import pandas as pd

df = pd.DataFrame({"age": [25, 30, 35], "salary": [50000, 60000, 70000]})
print(df.iloc[0])          # first row
print(df.iloc[0:2, 0:1])   # first 2 rows, first column
```

Output:

```
age          25
salary    50000
Name: 0, dtype: int64
   age
0   25
1   30
```

Boolean filtering (most-used ML pattern)

Filters rows using a True/False condition — the DataFrame equivalent of a SQL `WHERE` clause. This is probably the single most-used operation when exploring or cleaning a dataset.

```
import pandas as pd

df = pd.DataFrame({"name": ["Alice", "Bob", "Charlie"], "age": [25, 30, 35]})
print(df[df["age"] > 27])
print(df[(df["age"] > 20) & (df["age"] < 32)])   # multiple conditions
```

Output:

```
      name  age
1      Bob   30
2  Charlie   35
    name  age
0  Alice   25
1    Bob   30
```

query() — readable filtering

Same job as boolean filtering above, but written as a readable string expression instead of &/| operators — easier to read for filters with several conditions.

```
import pandas as pd

df = pd.DataFrame({"age": [25, 30, 35], "salary": [50000, 60000, 70000]})
print(df.query("age > 27 and salary < 70000"))
```

Output:

```
   age  salary
1   30   60000
```

4. Data Cleaning

isnull() / dropna() / fillna()

The core missing-data toolkit. `isnull` finds gaps, `dropna` removes incomplete rows, `fillna` fills gaps (with a constant, or a computed value like the column mean). Real-world datasets almost always need this before modeling.

```
import pandas as pd
import numpy as np

df = pd.DataFrame({"a": [1, np.nan, 3], "b": [4, 5, np.nan]})
print(df.isnull())
print(df.dropna())                    # drop rows with any NaN
print(df.fillna(0))                   # fill NaN with 0
print(df.fillna(df.mean(numeric_only=True)))   # fill with column mean
```

Output:

```
       a      b
0  False  False
1   True  False
2  False   True
     a    b
0  1.0  4.0
     a    b
0  1.0  4.0
1  0.0  5.0
2  3.0  0.0
     a    b
0  1.0  4.0
1  2.0  5.0
2  3.0  4.5
```

duplicated() / drop_duplicates()

Finds and removes exact duplicate rows, which can silently bias a model (e.g. by over-weighting a repeated sample) if left in the training data.

```
import pandas as pd

df = pd.DataFrame({"a": [1, 1, 2, 3], "b": ["x", "x", "y", "z"]})
print(df.duplicated())
print(df.drop_duplicates())
```

Output:

```
0    False
1     True
2    False
3    False
dtype: bool
   a  b
0  1  x
2  2  y
3  3  z
```

astype() — convert column types

Converts a column's dtype — very common right after loading a CSV, where numbers sometimes get read in as text and need converting before any math will work on them.

```
import pandas as pd

df = pd.DataFrame({"age": ["25", "30", "35"]})
df["age"] = df["age"].astype(int)
print(df.dtypes)
print(df)
```

Output:

```
age    int64
dtype: object
   age
0   25
1   30
2   35
```

replace()

Swaps specific values for others using a mapping dict — a simple manual way to encode categories as numbers, or to fix inconsistent category spellings (e.g. "NY" and "New York").

```
import pandas as pd

df = pd.DataFrame({"grade": ["A", "B", "A", "C"]})
df["grade"] = df["grade"].replace({"A": 1, "B": 2, "C": 3})
print(df)
```

Output:

```
  grade
0     1
1     2
2     1
3     3
```

5. Column Operations

apply() — apply a function

Runs a custom function over every value in a column (or row, with `axis=1`). Used for any transformation that isn't already a built-in Pandas method — the general-purpose escape hatch

for feature engineering.

```
import pandas as pd

df = pd.DataFrame({"salary": [50000, 60000, 75000]})
df["salary_k"] = df["salary"].apply(lambda x: x / 1000)
print(df)
```

Output:

```
   salary  salary_k
0   50000      50.0
1   60000      60.0
2   75000      75.0
```

map() — element-wise mapping on a Series

Like `replace`, but function- or dict-based element-wise substitution on a single Series — a common way to manually encode an ordinal category (low/medium/high) into numbers a model can use.

```
import pandas as pd

s = pd.Series(["low", "medium", "high"])
mapping = {"low": 0, "medium": 1, "high": 2}
print(s.map(mapping))
```

Output:

```
0    0
1    1
2    2
dtype: int64
```

assign(), rename(), drop()

`assign` adds new columns (returning a new DataFrame, useful for chaining), `rename` renames columns/index labels, `drop` removes columns or rows — the everyday DataFrame-shaping trio.

```
import pandas as pd

df = pd.DataFrame({"a": [1, 2, 3], "b": [4, 5, 6]})
df2 = df.assign(c=df["a"] + df["b"])
df2 = df2.rename(columns={"c": "sum_ab"})
df2 = df2.drop(columns=["b"])
print(df2)
```

Output:

```
   a  sum_ab
0  1       5
1  2       7
2  3       9
```

6. GroupBy & Aggregation

groupby() + agg()

Splits data into groups (e.g. by category), then computes a summary stat per group — the DataFrame equivalent of SQL GR0UP BY. Essential for exploring how a target variable differs across categories.

```
import pandas as pd

df = pd.DataFrame({
    "dept": ["Sales", "Sales", "IT", "IT", "HR"],
    "salary": [50000, 55000, 70000, 72000, 45000]
})
print(df.groupby("dept")["salary"].mean())
print(df.groupby("dept").agg({"salary": ["mean", "max", "min"]}))
```

Output:

```
dept
HR       45000.0
IT       71000.0
Sales    52500.0
Name: salary, dtype: float64
        salary
          mean    max    min
dept
HR     45000.0  45000  45000
IT     71000.0  72000  70000
Sales  52500.0  55000  50000
```

transform() — broadcast aggregation back to rows

Like `groupby().agg()`, but returns a result the same length as the original DataFrame — so you can attach a group statistic (e.g. "average salary for this person's department") as a new column on every row, a common feature-engineering trick.

```
import pandas as pd

df = pd.DataFrame({
    "dept": ["Sales", "Sales", "IT", "IT"],
    "salary": [50000, 55000, 70000, 72000]
})
df["dept_avg"] = df.groupby("dept")["salary"].transform("mean")
print(df)
```

Output:

```
    dept  salary  dept_avg
0  Sales   50000   52500.0
1  Sales   55000   52500.0
2     IT   70000   71000.0
3     IT   72000   71000.0
```

pivot_table()

Reshapes and summarizes data into a spreadsheet-style cross-tab (one category as rows, another as columns, values aggregated in the cells) — great for spotting interaction patterns during data exploration.

```
import pandas as pd

df = pd.DataFrame({
    "dept": ["Sales", "Sales", "IT", "IT"],
    "gender": ["M", "F", "M", "F"],
    "salary": [50000, 55000, 70000, 72000]
})
print(df.pivot_table(values="salary", index="dept", columns="gender", aggfunc="mean"))
```

Output:

```
gender        F        M
dept
IT      72000.0  70000.0
Sales   55000.0  50000.0
```

value_counts()

Counts how often each unique value appears in a column — the fastest way to check class balance in a classification target, or to see which categories dominate a feature.

```
import pandas as pd

s = pd.Series(["cat", "dog", "cat", "cat", "dog", "bird"])
print(s.value_counts())
print(s.value_counts(normalize=True))    # proportions
```

Output:

```
cat     3
dog     2
bird    1
Name: count, dtype: int64
cat     0.500000
dog     0.333333
bird    0.166667
Name: proportion, dtype: float64
```

7. Merging & Joining

merge() — SQL-style join

Combines two DataFrames based on a shared key column, just like a SQL JOIN. Needed whenever your features live in separate tables (e.g. customer info in one file, transactions in another) and need to be joined into one training table.

```
import pandas as pd

customers = pd.DataFrame({"id": [1, 2, 3], "name": ["Alice", "Bob", "Charlie"]})
orders = pd.DataFrame({"id": [1, 2, 2], "item": ["Book", "Pen", "Laptop"]})
```

```
print(pd.merge(customers, orders, on="id", how="inner"))
print(pd.merge(customers, orders, on="id", how="left"))
```

Output:

```
   id   name    item
0   1  Alice    Book
1   2    Bob     Pen
2   2    Bob  Laptop
   id     name    item
0   1    Alice    Book
1   2      Bob     Pen
2   2      Bob  Laptop
3   3  Charlie     NaN
```

concat() — stack DataFrames

Stacks DataFrames on top of each other (more rows) or side by side (more columns) without matching on a key. Common when combining multiple data batches or files with the same structure.

```
import pandas as pd

df1 = pd.DataFrame({"a": [1, 2]})
df2 = pd.DataFrame({"a": [3, 4]})
print(pd.concat([df1, df2], ignore_index=True))          # stack rows
print(pd.concat([df1, df2], axis=1))                     # stack columns
```

Output:

```
   a
0  1
1  2
2  3
3  4
   a  a
0  1  3
1  2  4
```

8. Sorting & Ranking

sort_values() / sort_index()

Reorders rows by column value(s) (`sort_values`) or by the index (`sort_index`) — used for things like ranking top predictions or presenting results in a sensible order.

```
import pandas as pd

df = pd.DataFrame({"name": ["Bob", "Alice", "Charlie"], "score": [85, 92, 78]})
print(df.sort_values("score", ascending=False))
print(df.sort_values("name"))
```

Output:

```
      name  score
1    Alice     92
```

```
0      Bob     85
2  Charlie     78
      name  score
1    Alice     92
0      Bob     85
2  Charlie     78
```

rank()

Assigns a rank number to each value in a column (handling ties automatically) — useful for turning a raw score into a relative ranking feature.

```
import pandas as pd

df = pd.DataFrame({"score": [85, 92, 78, 92]})
df["rank"] = df["score"].rank(ascending=False)
print(df)
```

Output:

```
   score  rank
0     85   3.0
1     92   1.5
2     78   4.0
3     92   1.5
```

9. DateTime

to_datetime() and .dt accessor

`to_datetime` converts text dates into a real datetime type; the `.dt` accessor then extracts parts like year/month/weekday. This is how raw date strings become usable numeric features (e.g. "is this a weekend?") for a model.

```
import pandas as pd

df = pd.DataFrame({"date": ["2024-01-01", "2024-02-15", "2024-03-20"]})
df["date"] = pd.to_datetime(df["date"])
df["year"] = df["date"].dt.year
df["month"] = df["date"].dt.month
df["day_of_week"] = df["date"].dt.day_name()
print(df)
```

Output:

```
        date  year  month day_of_week
0 2024-01-01  2024      1      Monday
1 2024-02-15  2024      2    Thursday
2 2024-03-20  2024      3   Wednesday
```

10. Stats & Correlation

describe(), corr(), cov()

`describe` gives per-column summary stats; `corr` gives the pairwise correlation matrix between numeric columns — the first thing you check to see which features actually relate to your target

variable before modeling.

```
import pandas as pd

df = pd.DataFrame({
    "hours_studied": [1, 2, 3, 4, 5],
    "score": [50, 55, 65, 70, 90]
})
print(df.describe())
print(df.corr())
```

Output:

```
       hours_studied      score
count       5.000000   5.000000
mean        3.000000  66.000000
std         1.581139  15.572412
min         1.000000  50.000000
25%         2.000000  55.000000
50%         3.000000  65.000000
75%         4.000000  70.000000
max         5.000000  90.000000
               hours_studied     score
hours_studied       1.000000  0.964579
score               0.964579  1.000000
```

11. ML Data Prep

get_dummies() — one-hot encoding

Converts a categorical column into multiple 0/1 columns (one per category) — most ML algorithms can't use text categories directly, so this is a standard required step before training on categorical features.

```
import pandas as pd

df = pd.DataFrame({"color": ["red", "blue", "green", "blue"]})
print(pd.get_dummies(df, columns=["color"]))
```

Output:

```
   color_blue  color_green  color_red
0       False        False       True
1        True        False      False
2       False         True      False
3        True        False      False
```

cut() / qcut() — binning continuous data

Converts a continuous numeric column into discrete buckets/categories (`cut` = custom-defined edges, `qcut` = equal-sized quantile buckets) — used to simplify a feature or create age-group / income-bracket style categories.

```
import pandas as pd

ages = pd.Series([5, 17, 22, 45, 68, 80])
```

```
bins = pd.cut(ages, bins=[0, 18, 35, 60, 100], labels=["child", "young", "adult", "senior"
print(bins)
```

Output:

```
0     child
1     child
2     young
3     adult
4    senior
5    senior
dtype: category
Categories (4, str): ['child' < 'young' < 'adult' < 'senior']
```

sample() — random sampling (quick train/test split idea)

Randomly selects a fraction of rows — the manual, from-scratch way to build a train/test split (in practice you'd normally use `sklearn.model_selection.train_test_split`, which does this plus a bit more).

```
import pandas as pd

df = pd.DataFrame({"x": range(10), "y": range(10, 20)})
train = df.sample(frac=0.8, random_state=42)
test = df.drop(train.index)
print("train size:", len(train), " test size:", len(test))
print(train.head(3))
```

Output:

```
train size: 8  test size: 2
   x   y
8  8  18
1  1  11
5  5  15
```

select_dtypes() — separate numeric vs categorical columns

Filters columns by data type — the standard way to split a dataset into numeric columns (which may need scaling) and categorical/text columns (which may need encoding) as a preprocessing setup step.

```
import pandas as pd

df = pd.DataFrame({"age": [25, 30], "name": ["Alice", "Bob"], "salary": [50000, 60000]})
print(df.select_dtypes(include="number").columns.tolist())
print(df.select_dtypes(include="object").columns.tolist())
```

Output:

```
['age', 'salary']
['name']
```

PART 3 — End-to-End Mini Pipeline

This ties NumPy + Pandas together the way a real ML preprocessing step looks: **load → clean → explore → encode → scale → array**.

```
import numpy as np
import pandas as pd

# 1. Simulate a raw dataset (in real life this comes from pd.read_csv)
np.random.seed(42)
df = pd.DataFrame({
    "age": np.random.randint(18, 65, 10),
    "salary": np.random.randint(30000, 120000, 10).astype(float),
    "city": np.random.choice(["Kolkata", "Delhi", "Mumbai"], 10),
})
df.loc[2, "salary"] = np.nan   # inject a missing value like real data

print("--- RAW DATA ---")
print(df)

# 2. Clean: handle missing values
df["salary"] = df["salary"].fillna(df["salary"].mean())

# 3. Explore
print("\n--- SUMMARY ---")
print(df.describe())
print("\n--- GROUPED BY CITY ---")
print(df.groupby("city")["salary"].mean())

# 4. Encode categorical column for ML
df_encoded = pd.get_dummies(df, columns=["city"])

# 5. Convert to NumPy array (what a model actually consumes)
X = df_encoded.drop(columns=["salary"]).to_numpy(dtype=float)
y = df_encoded["salary"].to_numpy()

# 6. Standardize features with NumPy (z-score), a near-universal ML step
X_scaled = (X - X.mean(axis=0)) / X.std(axis=0)

print("\n--- FEATURE MATRIX SHAPE ---")
print("X shape:", X_scaled.shape, " y shape:", y.shape)
print("\nFirst scaled row:", np.round(X_scaled[0], 3))
```

Output:

```
--- RAW DATA ---
   age    salary     city
0   56  117498.0    Delhi
1   46   74131.0    Delhi
2   32       NaN    Delhi
3   60   46023.0  Kolkata
4   25   71090.0  Kolkata
5   38   97221.0    Delhi
6   56   94820.0    Delhi
7   36   30769.0  Kolkata
```

```
8   40   89735.0  Kolkata
9   28   92955.0  Kolkata

--- SUMMARY ---
             age         salary
count  10.000000      10.000000
mean   41.700000   79360.222222
std    12.347289   25545.547134
min    25.000000   30769.000000
25%    33.000000   71850.250000
50%    39.000000   84547.611111
75%    53.500000   94353.750000
max    60.000000  117498.000000

--- GROUPED BY CITY ---
city
Delhi      92606.044444
Kolkata    66114.400000
Name: salary, dtype: float64

--- FEATURE MATRIX SHAPE ---
X shape: (10, 3)  y shape: (10,)

First scaled row: [ 1.221  1.    -1.   ]
```

Cheat Sheet — "I need to…" → method

Task	Method
Load data	`pd.read_csv()`
Peek at data	`df.head()`, `df.info()`, `df.describe()`
Handle missing values	`df.isnull()`, `df.dropna()`, `df.fillna()`
Filter rows	`df[df["col"] > x]`, `df.query()`
Select rows/cols by label/position	`df.loc[]`, `df.iloc[]`
Create/transform a column	`df["new"] = df["old"].apply(fn)`
Group + summarize	`df.groupby("col").agg(...)`
Combine tables	`pd.merge()`, `pd.concat()`
One-hot encode categories	`pd.get_dummies()`
Bin continuous data	`pd.cut()`, `pd.qcut()`
Convert DataFrame → array	`df.to_numpy()`
Reshape array	`arr.reshape()`
Elementwise math	ufuncs (`np.sqrt`, `np.exp`, `+ - * /` with broadcasting)
Normalize / standardize	`(x - x.min())/(x.max()-x.min())` or `(x - x.mean())/x.std()`
Matrix multiply	`A @ B` or `np.dot(A, B)`
Random shuffle / split prep	`np.random.permutation()`, `df.sample(frac=...)`
Linear algebra (PCA-adjacent)	`np.linalg.eig()`, `np.linalg.inv()`, `np.linalg.norm()`

Where to go next

- **scikit-learn**: `train_test_split`, `StandardScaler`, `LabelEncoder` formalize several patterns shown above.
- **Matplotlib / Seaborn**: visualize what you explored with `describe()`/`corr()`/`groupby()`.
- Practice by loading a real CSV (Kaggle has thousands) and running every method in this book against it once.